WeaveNYC — Multi-Cloud & PaaS Technical Note

| | |
|---|---|
| Project | WeaveNYC Midterm Proposal (Part 2) |
| Course | Cloud Computing — Spring 2026 |
| Prepared by | Kairan Shen |
| Date | March 9, 2026 |
| Purpose | Technical clarification of multi-cloud service design and PaaS usage |

This note provides additional technical detail on the multi-cloud design presented in the midterm proposal. WeaveNYC runs 8 managed cloud services across AWS, GCP, and Azure, plus Vercel for the frontend edge. Each provider was selected for a specific platform advantage — not for checkbox multi-cloud. The cloud supplements the local-first and P2P layers; it does not replace them.

## Section 1: Why the Cloud Still Appears in a Local-First / P2P Design

A local-first, P2P-aware application still needs the cloud where a browser-based peer network is weak: global API ingress, searchable geo-indexes, event-driven compute, notifications, and speech processing. The cloud fills these gaps without owning the message plane.

The boundary is simple: private state stays on device (IndexedDB), community events travel through the GossipSub peer overlay, and the cloud provides managed compute and searchable infrastructure around this core. If ordinary community events were routed through a mandatory cloud queue in normal operation, the P2P claim would no longer hold.

## Section 2: Service Map — Provider by Provider

The proposal decomposes the backend into 6 microservices, each deployed to the provider that best fits its workload characteristics.

| Provider / Service | WeaveNYC Component | Service Model | Why This Provider | Key Config |
|---|---|---|---|---|
| AWS Lambda | API Gateway + Match Engine + Posts Service | FaaS (strong PaaS) | Per-request scaling, no server management, native integration with RDS and API Gateway | Node 20 runtime, pnpm 9, GitHub Actions deploy |
| Amazon API Gateway | HTTP/REST/ WebSocket front door | Managed API platform | Managed ingress with rate limiting, auth, and WebSocket support for chat | Routes to Lambda + Fargate |
| Amazon RDS (PostgreSQL + PostGIS) | Central relational and geospatial index | DBaaS | PostGIS enables spatial queries: ST_DWithin(geom , point, | postgis/postgis:16-3.4, daily backup to S3 (30-day retention) |

| | | | | |
|---|---|---|---|---|
| | | | radius_meters) for "resources within 2 miles of this ZIP" | |
| Amazon ElastiCache (Redis) | Session, rate limiting, Pub/Sub, task queue | Managed cache | Redis 7 for token bucket rate limiting (@fastify/rate-limit), BullMQ task queues, chat Pub/Sub | Redis 7-alpine in dev |
| AWS Fargate (ECS) | Chat Service — persistent WebSocket connections | Serverless containers | Lambda has a 15-minute execution limit — WebSocket chat needs long-lived connections that Fargate provides | Container-based, auto-scaling on CloudWatch metrics |
| Google Cloud Run | Resource Directory Service — ingestion, parsing, geo-query | Fully managed containers (strong PaaS) | Scale-to-zero: resource lookups spike unpredictably. Isolating this service on GCP prevents spikes from taking down the main AWS backend. Blast radius decision. | Stateless container, deploy via GitHub Actions |
| Google Cloud Speech-to-Text | Tier 3 voice input — multilingual speech recognition | Managed AI API | The proposal needs speech-to-text as a service (en/es/zh), not a self-hosted ASR stack. Tier 1 and 2 use device-native and Web Speech API first. | Max 20s audio, WebM/OGG/WAV input |
| Azure Functions | Notification Service — push/SMS/email orchestration | FaaS (strong PaaS) | Event-driven notification workflows. Separating notifications on Azure provides isolation from the core API and satisfies the course multi-cloud requirement. | Trigger-based, deploy via GitHub Actions |
| Azure Functions (2nd workload) | Speech Gateway — orchestration for voice accessibility | Same platform, separate workload | Routes audio to the appropriate speech provider based on the three-tier degradation model (native → Web Speech API → Cloud) | Single endpoint: POST /v1/speech/transcribe |

| | | | | |
|---|---|---|---|---|
| Vercel Edge | PWA frontend + Anchor Admin Console | Edge platform / CDN | Native Next.js support, global CDN, instant rollback (<1 min). Two separate Next.js apps: user-facing PWA and Anchor management console. | staging.weavenyc.org / app.weavenyc.org |

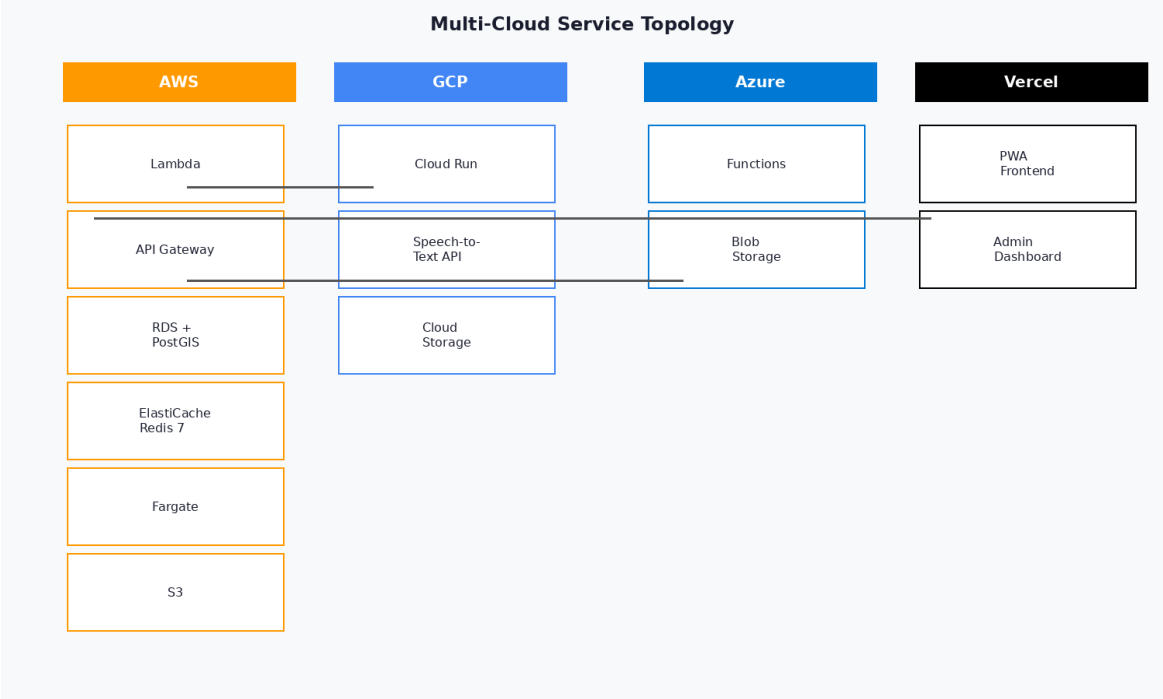



*Figure 1: Multi-Cloud Service Topology*

Section 3: Strongest PaaS Examples for the Course

If asked which services best demonstrate platform-level cloud understanding:

Unambiguous PaaS/FaaS: AWS Lambda, Google Cloud Run, Azure Functions. The provider manages servers, scaling, and infrastructure. The student supplies code, container images, and configuration.

Strong managed platforms: Amazon RDS (PostgreSQL + PostGIS) as DBaaS, Amazon API Gateway as managed ingress, Google Cloud Speech-to-Text as managed AI capability.

Useful but category-gray: AWS Fargate. Operationally valuable for WebSocket workloads that exceed Lambda's 15-minute limit, but serverless containers sit between IaaS and PaaS in strict classical taxonomy. The proposal does not rely on Fargate alone to make the PaaS argument.

Section 4: Why Multi-Cloud — Specific Justifications

Each cloud solves a different problem. This is not multi-cloud for its own sake.

| Provider | Primary Role | Architectural Reason |
|---|---|---|
| AWS | Primary backend — API, database, cache, chat | RDS PostGIS provides managed spatial indexing for the core matching use case: "resources within 2 miles of this ZIP." Lambda and Fargate handle the full API lifecycle. |
| GCP | Isolated resource directory + speech | Cloud Run's scale-to-zero behavior isolates unpredictable resource lookup spikes from the main backend. If the resource directory experiences a traffic surge, the AWS API stays unaffected. This is a blast radius decision. |
| Azure | Notifications + voice gateway | Event-driven notification workflows (push/SMS/email) are a natural fit for Azure Functions. Separating notifications reduces coupling with the core API and provides a clean isolation boundary. |
| Vercel | Frontend edge delivery | Native Next.js deployment with global CDN. Instant rollback (<1 min) for frontend incidents. |

The cost of multi-cloud is real: more credentials, more monitoring edges, more deployment complexity. The justification holds only because the interfaces remain simple (REST APIs between services) and the MVP can be reduced to fewer providers if time requires simplification.

Section 5: Data Placement and Privacy Boundary

Three storage zones, each with a clear privacy boundary:

| Zone | Storage Technology | What Lives Here | Privacy Rule |
|---|---|---|---|
| Local-only (Tier 1) | IndexedDB via Dexie.js | Drafts, preferences, Ed25519 keys, post cache, bookmarks, device ID | Never leaves the device. Never synced to server. |
| Community-shared (Tier 2) | IPFS Helia (content-addressed) + GossipSub | Post summaries (PostSummaryDAG), version chains (CID → prev_cid), reputation signals, moderation events | Replicated across peers. Geofenced by community — a Bronx node only stores Bronx data. Ed25519 signed for tamper-proofing. |
| Cloud index (Tier 3) | PostgreSQL + PostGIS, Redis, S3 | Resource directory, anonymized post index, match results, notification jobs, | Anonymized only. Phone/email stored as SHA-256 hash. No user GPS. Encrypted at rest. |

| | | | |
|---|---|---|---|
| | | analytics events | Every query geofenced by ZIP. |

Cloud usage does not imply centralization of user data. The cloud indexes what it needs for search, matching, and notifications — it does not hold a full copy of every private user context.

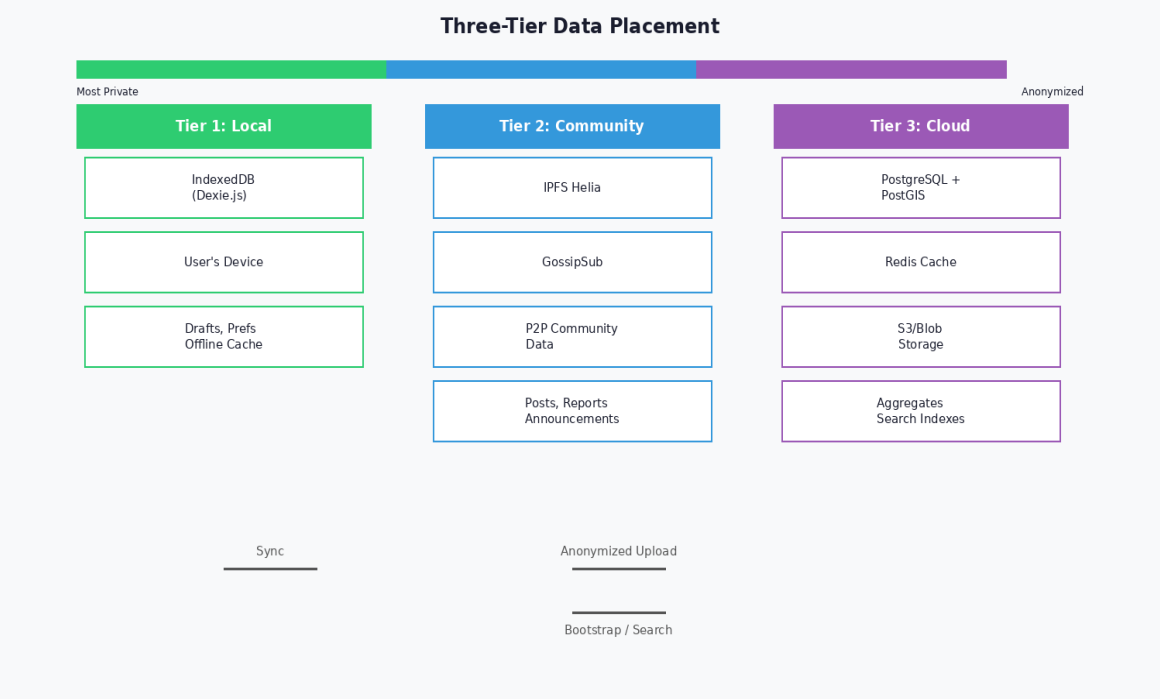



*Figure 2: Three-Tier Data Placement Model*

Section 6: Operational Configuration

Key operational parameters from the deployment design:

| Parameter | Value |
|---|---|
| Node version | Node 20 (node:20-alpine Docker base) |
| Package manager | pnpm 9 (via corepack) |
| CI/CD | GitHub Actions — lint/typecheck/unit on feature branches, full suite on PR, auto-deploy staging on dev merge, manual approval for production |
| Database | PostgreSQL 16 + PostGIS 3.4 (postgis/postgis:16-3.4) |
| Cache | Redis 7 (redis:7-alpine) |
| API framework | Fastify + TypeScript (auto-generated Swagger docs) |
| ORM | Drizzle (type-safe SQL) |
| Monorepo | pnpm workspace (shared types and tools) |

SLO Targets

| Metric | Target |
|---|---|
| Home search (p95) | < 800ms |
| Post creation (p95) | < 1s |
| Match generation (p95) | < 2s |
| API availability | >= 99.5% (monthly) |
| First screen load (p95) | < 3s on 4G |
| Error budget | 3.6 hours/month |

Rate Limiting (Redis token bucket via @fastify/rate-limit)

| Endpoint | Limit |
|---|---|
| Anonymous generic | 30 req/min |
| Logged-in generic | 120 req/min |
| Send message | 20/min |
| Post create | 5/min |
| Speech transcribe | 10/min |
| Report create | 5/min |

Monitoring stack: Prometheus + Grafana (metrics), Loki (logs), OpenTelemetry + Jaeger (tracing), Sentry (errors), UptimeRobot (uptime).

Section 7: Rollback and Recovery

| Component | Method | Target Time |
|---|---|---|
| Frontend (Vercel) | Instant Rollback | < 1 minute |
| AWS Lambda | Alias revert | < 2 minutes |
| GCP Cloud Run | Traffic split | < 2 minutes |
| Azure Functions | Deployment slot swap | < 2 minutes |
| Database | Migration rollback / backup restore | 5-30 minutes |

Database backup: daily pg_dump to S3, 30-day retention. Resource directory snapshots are permanently versioned in Cloud Storage. RPO: 24h for PostgreSQL, 0 for resource directory.

Section 8: Summary

WeaveNYC uses provider-managed application platforms rather than a VM-first deployment. The strongest PaaS examples are AWS Lambda, Google Cloud Run, and Azure Functions — all three let the provider manage servers and scaling while the student focuses on code and architecture. Supporting services (RDS/PostGIS, API Gateway, ElastiCache, Cloud Speech-to-Text) make the system practical without forcing the semester into an operations exercise. The multi-cloud design is justified by specific platform advantages and blast radius isolation, not by provider count. The cloud supplements the local-first and P2P core — it indexes, enriches, notifies, and assists, but it does not own the message plane.

References

[1] NIST SP 800-145, The NIST Definition of Cloud Computing

[2] AWS Lambda documentation — https://docs.aws.amazon.com/lambda/

[3] Google Cloud Run documentation — https://cloud.google.com/run/docs

[4] Azure Functions overview — https://learn.microsoft.com/en-us/azure/azure-functions/

[5] Amazon RDS for PostgreSQL — https://aws.amazon.com/rds/postgresql/

[6] PostGIS documentation — https://postgis.net/docs/

[7] AWS Fargate documentation — https://docs.aws.amazon.com/fargate/